

MINI-AIOS

New Joiner Complete Guide

From zero to your first published dashboard — one self-contained guide, no gatekeepers required.

Prepared for	Every new staff member joining Mini-AIOS
Maintained by	Varmens
Teams ID (Varmens DigitWeb)	varmensk.digitweb@gmail.com
New joiners	Contact Varmens directly for setup help

Read this guide start to finish, in order. Do each setup step as you reach it — don't skip ahead.

Table of Contents

1. Introduction	3
2. How We Work — Read This Before Anything Else	3
3. Understanding Our Business.....	4
4. Operating Principles.....	5
5. One-Time Setup — Do This Once, On Day One	5
6. Standard Folder Structure — Reference.....	7
7. Our Databases — What's Where	8
8. Your Task Workflow	9
9. Publishing Your Output	11
10. Your Daily Work Log.....	13
11. Taking Over Someone Else's Task	14
12. Governance Essentials — Keep These in Mind.....	15
13. Troubleshooting	15
14. Quick Reference Card	16

1. Introduction

1.1 What This System Is

You are joining a working model built around two AI tools with two different jobs:

- ChatGPT is the Brain — it plans, clarifies your requirement, and writes the exact instructions Claude Code will run.
- Claude Code is the Worker / Execution layer — it inspects files, builds folders, writes code, pulls data, and creates evidence. It executes; it does not decide what to build.

GitHub, PostgreSQL, and the evidence files inside your own folders are the proof layer — what actually happened, saved where anyone can check it later.

1.2 The Shape of Your Work, End to End

ONE-TIME SETUP	→ GitHub account, your repo, your standard folder skeleton
EVERY NEW TASK	→ GPT clarifies + writes prompts → Claude Code executes
EVERY TASK OUTPUT	→ HTML view → pushed to the URL your team lead assigns
EVERY WORKING DAY	→ one row logged to your <code>daily_task</code> table

1.3 Who To Ask

Role	Person	Ask them about
Team Leader / Trainer	Varmens	Setup help, database credentials, where to publish, anything unclear in this guide
Technical Reviewer	Sajeesan	Technical correctness of what Claude Code built
Queryability Reviewer	Tamil Selvan	Whether your evidence/documentation is clear enough for someone else to reuse
Managing Director	MD	Final approval on anything outside your assigned scope

Teams ID (Varmens DigitWeb): varmensk.digitweb@gmail.com — new joiners, contact Varmens directly.

2. How We Work — Read This Before Anything Else

2.1 There Is No Fixed Workflow

The single most important thing to understand before you start: our work does not follow a static or permanently fixed process. It changes — sometimes weekly, sometimes monthly — as we find better ideas and bring in new technology from the industry.

This guide describes how things work right now. Expect parts of it to be updated as the workflow evolves. Your job is not to memorize one fixed process — it's to stay adaptable and comfortable picking up new tools and methods as they're introduced.

2.2 What Stays Constant

While the specific tools and steps change, the underlying principles don't:

- GPT plans, Claude Code executes
- Check for existing work before building something new
- Every completed task leaves evidence behind
- Folder structure and documentation stay consistent so anyone can pick up anyone else's work

3. Understanding Our Business

3.1 What Kind of Company This Is

We are not a client-services company and not a software product company. We are an e-commerce company — we design, source, and sell our own lighting products directly to customers, through our own website and through third-party marketplaces.

That distinction matters for how you should think about your work: there is no external client to satisfy and no software product to ship. The “customer” of most of what you build is the business itself — the people making day-to-day decisions about stock, pricing, ads, and sales.

3.2 E-Commerce Basics — If You're New To This

A few terms you'll see constantly. This is a plain-language starting point, not the full picture — ask Varmens for specifics on our current setup.

Term	What it means
SKU	Stock Keeping Unit — a unique code identifying one specific product (and often one specific variant, like a color or size). Nearly everything else — stock, sales, listings — is tracked against a SKU.
Marketplace	A third-party platform where our products are listed and sold alongside other sellers' products (for example, sites like Amazon or eBay). We don't control the platform itself, only our listings on it.
Platform	More broadly, any online storefront or system our products are sold through — including our own website, not just third-party marketplaces.
Brand	The product brand name(s) our SKUs are grouped under — distinct from the marketplace they're sold on. One brand can be sold across many marketplaces.
Sales data	Records of what sold, when, at what price, and through which platform — the raw material for almost every dashboard you'll build.
Ads	Paid advertising campaigns (e.g. marketplace PPC/sponsored listings) run to drive traffic and sales — tracked separately from organic sales so their cost and return can be measured.

3.3 How This Maps to Our PostgreSQL Data

Each of these concepts usually corresponds to a table, or a set of columns, somewhere in our PostgreSQL databases — a SKU/product table, an orders/sales table, a marketplace or platform reference table, an ads-spend table, and so on.

You are not expected to know the exact layout from memory. Section 7 explains which database holds what at a high level, and every task you're assigned should come with (or you should request) a database-structure file telling you exactly which tables and columns are relevant to that specific task.

4. Operating Principles

4.1 Efficiency First — No Hardcoding

We enforce efficiency. Hardcoding values, one-off scripts, and manual shortcuts that bypass the standard workflow are not useful here and are not recommended — there's no need for them.

This applies equally to everyone. Whether you're an intern or a senior developer, you're expected to fully adapt to a workflow that is completely AI-based, using GPT and Claude Code as described in this guide rather than working around them.

4.2 You Are a Solution Provider

Your role isn't just to complete assigned tickets. If you spot a problem yourself, or another staff member brings you one, it's your job to provide a solution — using the current workflow, or a new idea of your own, with AI support.

Whatever you build or handle — a system, a task, a one-off fix — should have one underlying objective: solve the problem, provide a working solution, and help scale up sales for the company.

5. One-Time Setup — Do This Once, On Day One

Nobody needs to build this for you. Work through 5.1 → 5.6 in order, doing each step as you reach it — don't just read ahead and come back later.

5.1 Create Your GitHub Account

1. Use your company office Gmail address to sign up at github.com — not a personal email.
2. Confirm the account and note down the exact username — you will use it constantly.
3. Tell Varmens your GitHub username so you can be added to the company organization/repo access where required.

5.2 Create Your Repository

Create your own repository (or your own top-level folder inside the assigned company repo, if Varmens tells you one already exists for your team — ask if unsure). Name it clearly with your name or team, no vague names like “test” or “new.”

5.3 Install Your Tools

Pick one of the following — whichever you already know, or whichever Varmens recommends if you're starting from zero:

Option	Best if...
VS Code + Claude Code extension/terminal	You want a familiar code-editor feel with the Source Control panel as your Git GUI
Claude Code Desktop app	You want the simplest standalone experience, no separate editor
Claude Code in terminal / cmd	You're comfortable working directly in a command line

All three run the same Claude Code — pick based on comfort, not capability. You can switch later without losing anything, since all your real work lives in your files and GitHub, not inside the tool.

5.4 Check Your Connectors

Before you start using Claude or Claude Code for any real work, confirm these connectors are enabled in both Claude and Claude Code:

- postgres — read access to Developer 1's database (Varmen AIOS, Hub, static HTMLs)
- LEDSone MCP — read access to the LEDSone database (varmen_db)
- LEDSone MCP DOC — documentation/reference connector for the LEDSone database

If any of these are missing, ask Varmen before starting — don't attempt to add your own database credentials as a workaround (Section 7.3).

5.5 Build Your Folder Skeleton

This is done through the same GPT → Claude Code loop you'll use for every task from now on — so it doubles as your first practice run. Do it now, before moving to Section 6.

1. Open a brand-new ChatGPT chat (no special project needed).
2. Paste the prompt below, filling in your name and repo URL.
3. GPT will hand you a Claude Code DISCOVERY prompt. Copy it exactly into Claude Code — do not type your own instructions.
4. Paste Claude Code's discovery findings back into the same GPT chat. GPT checks nothing conflicting already exists.
5. GPT then hands you a Claude Code BUILD prompt. Run it — it creates all standard folders plus a short README.md in each explaining its purpose, and pushes the result to GitHub.
6. Paste Claude Code's “what I built” output back into GPT for a final check against the folder standard (Section 6). Fix anything GPT flags before moving on.

 **Paste this into a new ChatGPT chat**

I'm a new staff member setting up my Mini-AIOS working folder for the first time.

My name: [Your Name]

My GitHub repo: [repo URL]

Act as my planning brain for this setup (GPT = Brain, Claude Code = Worker). Please:

1. Generate a Claude Code DISCOVERY prompt to check my GitHub repo for any existing folder structure or files, so I don't create duplicates.
2. After I paste back what Claude Code finds, review it and generate a Claude Code BUILD prompt that creates the full standard skeleton: evidence/, documentation/, handover/, closure/, validation/, workflows/, sql/, capability/, prompts/, data-maps/, query-packs/, duplicate-risk-reports/ – with a short README.md in each folder explaining its purpose, then pushes the result to GitHub.
3. After I paste back what Claude Code built, review it against the folder standard and tell me PASS or exactly what's missing.

Do not let me type build instructions directly into Claude Code – always generate the exact prompt for me to paste.

5.6 Setup Checklist — Confirm All Before Starting Real Work

- GitHub account created with office Gmail
- Repository created and accessible
- One tool installed and working (VS Code / Claude Code Desktop / Claude Code CLI)
- postgres, LEDSone MCP, and LEDSone MCP DOC connectors all enabled
- All standard folders exist, each with a README.md, pushed to GitHub
- You know where the shared task Google Sheet is, and can see your row when one is assigned

6. Standard Folder Structure — Reference

Identical for every staff member. Only the files inside change to match your actual work.

Folder	What goes in it
evidence/	Proof that work happened — query outputs, logs, screenshots, exported results
documentation/	Explanations of what something is, why it exists, how to use it
handover/	Notes so someone else can pick up unfinished work without asking you
closure/	End-of-task / end-of-day closure notes
validation/	Checklists and reports proving output is correct
workflows/	Documentation of any automation (scheduled regeneration, scripts, etc.)
sql/	SQL files — inspection queries, validation queries, any SQL you wrote

Folder	What goes in it
capability/	Reusable methods discovered while working — patterns worth reusing later
prompts/	Saved copies of the GPT-generated Project Instructions and SKILL.md for each task
data-maps/	Source-to-target mapping, including the PostgreSQL database-structure file for each task
query-packs/	Grouped, reusable sets of queries for a recurring purpose
duplicate-risk-reports/	Findings from Existing-Asset-First checks before creating something new

Per-task subfolders

Every folder except capability/ follows the pattern [folder]/[task-name]/ — e.g. evidence/july-inventory-check/. capability/ stays flat because capabilities are reused across tasks, not owned by one.

Forbidden folder/file names

Never use: test, final, new, temp, random, notes, old. These hide what something actually is and make it un-queryable later.

6.1 SKILL.md vs. CLAUDE.md — Two Different Files

Every task ends up with both of these saved — they serve different readers and shouldn't be confused for one another:

File	Written for	Purpose
SKILL.md	GPT	A cut-down version of our governance rules (Existing-Asset-First, Evidence Rule, Duplicate-Truth Prevention, etc.), scoped to this specific task. Uploaded as a Source into the task's GPT Project so GPT stays disciplined while writing prompts.
CLAUDE.md	Claude Code	The standard file Claude Code automatically looks for when it opens a folder. A short briefing on what this task/project is, so Claude Code — and anyone who opens Claude Code in that folder later — has context without being told out loud.

7. Our Databases — What's Where

We currently run two separate PostgreSQL databases, built by two different developers. Knowing which is which — and who to ask for write access — will save you a lot of confusion.

Database	What's in it	Read access	Write access
postgres (Developer 1)	Varmen AIOS, the Hub, and static HTML outputs. Not yet migrated — currently kept as a backup.	postgres MCP connector	temp_user credential (ask Varmens) — covers Varmen AIOS and PH task pushes
LEDSone (Developer 2)	varmen_db — our currently active working database	LEDSone MCP connector	Separate credential, issued by Varmens on request

7.1 If Your Task Needs Write Access

The default answer is varmen_db. Contact Varmens for write-access credentials — don't assume you already have them, and don't reuse credentials from a previous task without checking they're still current.

7.2 Where Your Finished Output Gets Published

This is decided per task by your team lead (Varmens), not fixed by this guide — it could be the existing Varmen AIOS/Hub, a newly created destination, or a migration into varmen_db. Always confirm before publishing rather than assuming (Section 9).

7.3 Never Share Passwords With Claude or GPT

Strict rule, no exceptions

Do not paste database passwords, connection strings, or any other credentials into a Claude or GPT conversation, prompt, or file that gets committed anywhere. Connectors are configured outside the chat for exactly this reason — use them instead of typing credentials in.

8. Your Task Workflow

8.1 The Current Workflow, In Short

- Maintain your folder structure cleanly, following the standard folders (Section 6)
- GPT = Brain, Claude Code = Execution
- Task requirements arrive as a Google Sheet with a self-explanatory README tab, the data table needed, 3–5 reference data examples, and column descriptions
- You generate a static HTML file as a table view of the required data
- You push that static HTML to the URL your team lead provides
- Our database is PostgreSQL, reachable from Claude Code / your Claude account via MCP — all our data lives there

8.2 Task Requirement Format

Every requirement you receive, from anyone, should be a Google Sheet containing:

Tab / Section	Contents
README tab	Self-explanatory description of what's being requested and why
Data table needed	The exact table/columns the requester wants to see
Reference examples	3–5 sample rows of real or representative data, so the shape is unambiguous
Column descriptions	What each column means — no guessing required

8.3 The Standard Loop

1. Create a new GPT Project for the task, with:
 - the Project Instructions
 - the task requirement file (from the Google Sheet)
 - a PostgreSQL database-structure file (which data lives where)
2. Ask GPT to discover the database for similar or related tasks first (Existing Asset First – don't rebuild something that already exists)
3. Get the standard folder-creation prompt from GPT → run it in Claude Code
4. Claude Code auto-saves task files as it works: handover/, evidence/, prompts/, plus a CLAUDE.md and a README for the task
5. Finish: generate the HTML view → push it to the assigned URL → push the work to your GitHub repo

 *Paste this into a new ChatGPT chat to kick off any new task*

I have a new task requirement from the team (from our shared Google Sheet, with a README tab, the data table needed, reference examples, and column descriptions). Act as my planning brain for this task (GPT = Brain, Claude Code = Worker).

Task requirement (pasted from the sheet):
[paste the requirement details here]

PostgreSQL database-structure notes (if I have them):
[paste or describe which database/tables are relevant, if known]

Please:

1. Ask me anything you need to clarify the requirement, scope, and expected output.
2. Tell me how to discover whether a similar or related task already exists in our database or folders, so we don't duplicate it.
3. Generate the Project Instructions for a new dedicated GPT project for this task.
4. Generate a task-specific SKILL.md, adapted only from the standard Mini-AIOS rules, scoped to this task.
5. Generate the standard folder-creation prompt to run first in Claude Code, followed by the first DISCOVERY prompt.

I will create a brand-new GPT project for this task, paste in your Project Instructions and this SKILL.md as the Source, and only then start running Claude Code prompts from that project.

8.4 Automating Your Output

Once the task's HTML view is working, add automation so it refreshes on its own — daily, weekly, or monthly, based on how often the underlying data actually needs to update. Manual regeneration is the fallback, not the default, once a task is in steady use.

8.5 The First-Few-Tasks Rule

Stage	Rule
Your first few tasks	Always go GPT → prompt → Claude Code. No exceptions while you're learning the pattern.
Once you know the pattern	You may type prompts directly into Claude Code without a fresh GPT round-trip — but going through GPT first remains the recommended default, especially for anything non-trivial.

8.6 Task Closure

Before calling a task done, your closure note (saved in closure/) should answer:

- What was the requirement?
- What asset/evidence exists, and where?
- What is the GitHub path or commit?
- Can someone else continue this without asking you? (Section 11)
- What's the one next step?
- PASS or FAIL?

9. Publishing Your Output

Pulling and analyzing data doesn't change — keep using whatever connector you already use for that. This section covers only the last step: getting your finished HTML live at the URL your team lead assigns.

Your team lead decides the destination

Varmens decides, per task, exactly where your output should be published — the existing Varmen AIOS/Hub, a new destination, or a migration into varmen_db. Always confirm before publishing rather than assuming which one applies.

9.1 If You're Assigned the Varmen AIOS / Hub

Required fields, every time, no exceptions:

Field	Rule
member_name	Always your own name, spelled identically every time — lowercase, e.g. apirame, never Apirame or a nickname
page_slug	Short, URL-safe, unique to you, e.g. july-inventory-check. Re-using a slug updates that dashboard instead of duplicating it
page_title	Human-readable name shown on the Hub

Field	Rule
html_content	One complete, self-contained HTML file — inline all CSS/JS, no external <script src> or <link> references

1. Build and finish your dashboard exactly as normal; sanity-check it renders correctly as a standalone file first.
2. Save the finished HTML to a file on disk — don't paste large HTML directly into a Claude Code prompt.
3. Ask Varmens for the current push script and connection string for this destination.

```
# Git Bash / macOS / Linux:
export HUB_DB_URL="<value Varmens gives you>"
node push_to_hub.js --member "your_name" --slug "your-dashboard-slug" \
  --title "Your Dashboard Title" --file "./your-dashboard.html"
```

On plain PowerShell, \$env: syntax works differently with this script — ask Varmens; Git Bash is the tested path.

9.2 If You're Assigned the PH Team Board

PH is a team name. Their tool reads task rows from tech_team_outputs.ph_task — you (the developer) write a row as HTML; the PH end user sees it, acts on it, and their action is recorded back onto the same row.

Column	Who fills it / meaning
project_name, project_code, task_name	You — required, identify the task
task_id, team, developer	You — team = the responsible team, developer = you
assigned_user	You — the PH end user expected to act
html_content	You — the task body as HTML, shown in the tool
description	You — free-text detail
phase_level, version_level, version_status	You — defaults to 0 / 0 / blank; set if your task tracks phases or versions
action_took_by, action_took_date_time	Left NULL by you — filled in when the task is completed
created_at, updated_at	Auto — never fill manually

9.3 If You're Migrating Into varmen_db

Ask Varmens for the current write credentials and target table/schema before pushing anything new here — this destination is actively used for day-to-day work, so confirm naming with Varmens to avoid clashing with existing data.

9.4 Checklist Before Any Push

- HTML is fully self-contained (opens correctly as a standalone file)

- You've confirmed the destination with Varmens — not assumed it
- Naming (member_name / slug / project_code, as relevant) matches your prior convention
- Nothing sensitive / internal-HR-only is in output meant for a wider audience

10. Your Daily Work Log

Separate from publishing output. This is where every developer records what they did each working day, so it becomes permanent, searchable company knowledge instead of living only in chat or memory.

Different login required

The daily_task schema is not reachable with the temp_user or varmen_db credentials — you need a separate, distinct login for this. Ask Varmens; it will be provided separately.

10.1 Your Table

Every developer has their own table per project:

```
daily_task.tbl_<projectcode>_<developer>
e.g. tbl_invmgt_arun, tbl_wlsp_sarujanan
```

One row = one activity you completed on one day. The table is append-only: you INSERT a new row for each new activity — you never overwrite or delete a previous day's rows.

10.2 The 8 Fields You Always Fill

Field	Meaning
activity_id	Unique ID, format D<day>-A<seq>, e.g. D08-A01. Check your last used ID before adding a new one.
activity_date	Date the work was done, YYYY-MM-DD
developer	Your username
project_code	Short project code, matches your table name
activity_type	development / devops / analysis / publication / bugfix / review / documentation
activity_title	Short one-line title
activity_summary	What you did, how, and the outcome — written for a stranger to understand
status	completed / in_progress / blocked / planned

Everything else in the table (systems_touched, evidence_refs, next_action, memory_tags, etc.) is optional but recommended — the more you fill, the more useful and searchable your record is. imported_at, created_at, and updated_at fill themselves — never touch them.

10.3 Step by Step

1. Find your last activity_id: SELECT ... ORDER BY created_at DESC LIMIT 5, so your new one is unique.
2. Fill in the 8 mandatory fields at minimum.

3. Run one INSERT per activity.
4. Verify: SELECT ... WHERE activity_id = '...' must return exactly one row, with created_at showing just now.

```
INSERT INTO daily_task.tbl_<projectcode>_<developer> (
  activity_id, activity_date, developer, project_code,
  activity_type, activity_title, activity_summary, status
) VALUES (
  'D01-A01', '2026-01-15', 'arun', 'INVMGT',
  'development', 'Built the low-stock alert query',
  'Wrote and tested a SQL query flagging SKUs below reorder point.',
  'completed'
);
```

10.4 Common Mistakes

Mistake	Fix
Duplicate activity_id	Run the last-ID check first (10.3, step 1)
UPDATE-ing an old row instead of INSERT-ing	Always INSERT a new row — history must stay intact
Vague activity_summary (“did some work”)	Write what, how, and the result — assume the reader wasn't there
Filling created_at/updated_at manually	Leave them out — they auto-fill
Booleans as text ('true')	Use unquoted true / false

10.5 Golden Rule

Log every working day, at end of day

One activity = one row. Never rewrite history — correct a mistake with a new corrective row, not a delete. Verify after every insert.

11. Taking Over Someone Else's Task

Because the whole team maintains the same folder structure and file conventions, taking over a task from another member should be simple — no meeting required.

1. Clone the task's folder to your own PC.
2. Open Claude Code inside that folder.
3. Ask it a simple question — Claude Code will read the folder's CLAUDE.md, README, evidence, and handover notes and explain the rest.

 **Paste this into Claude Code once you're in the folder**

From today, I have to take over this task. Can you explain to me what it is about, what I need to do, where to begin, what has been done, and what needs to be achieved?

If Claude Code can't answer this clearly, that's a signal the previous owner's handover/evidence was incomplete — flag it to Varmens rather than guessing.

12. Governance Essentials — Keep These in Mind

These rules don't change task to task, even while the specific tools and steps around them do. They're what keeps your work trustworthy and reusable by anyone who looks at it later.

12.1 GREEN / AMBER / RED

Level	Meaning	Example
GREEN	Safe to do without extra approval	Documentation, evidence packs, read-only inspection, safe Claude Code prompts
AMBER	Needs reviewer approval first	Draft SQL later run by a developer, workflow documentation updates, config documentation
RED	Not allowed without written approval	Production data changes, schema drops, business-rule changes, live automation execution

12.2 Evidence Rule

No evidence = no completed work. “It works,” “I checked manually,” and “I’ll upload later” are not evidence. A Git path, a saved query result, or a saved file is.

12.3 Existing Asset First

Before creating anything new — including a database object — check what already exists in your folders, GitHub, and the relevant database. Reuse → extend → merge → create new, in that order.

12.4 The Unknown Developer Test

Before closing any task, ask: if someone who has never seen this opened your repo tomorrow, could they understand what it is, why it exists, what's done, what evidence proves it, and what to do next — without asking you? If not, it isn't finished yet.

12.5 Never Share Credentials With AI

Repeated here because it matters: never type or paste passwords, connection strings, or any other credentials into Claude or GPT, in any form. Use the configured connectors instead.

13. Troubleshooting

Problem	Likely cause / fix
Claude Code seems to be deciding what to build on its own	Stop. Go back to GPT for the next prompt — Claude Code should only execute, never plan (unless you've passed your first-few-tasks stage, and even then GPT-first is recommended)

Problem	Likely cause / fix
Can't connect to a database from Claude Code	Check the postgres / LEDSone MCP / LEDSone MCP DOC connectors are enabled (Section 5.4)
Not sure where to publish a finished dashboard	Ask Varmens — don't assume; the destination is decided per task (Section 9)
Push script fails on PowerShell	Use Git Bash instead, or ask Varmens — \$env: syntax isn't the tested path
Re-pushing created a duplicate instead of updating	You used a different slug/identifier than last time — reuse the exact same one to update in place
daily_task INSERT fails: duplicate key	Run the last-activity_id SELECT first, then pick the next unused ID
Not sure if something already exists before building it	That's an Existing-Asset-First discovery prompt — ask GPT to generate one before you create anything, including a database check
Taking over a task and Claude Code can't explain it	The previous owner's handover was incomplete — flag it to Varmens rather than guessing

14. Quick Reference Card

SETUP (once) : GitHub (office Gmail) → repo → tool → connectors → folder skeleton
 DATABASES : postgres (Varmen AIOS/Hub, backup) | LEDSone (varmen_db, active)
 WRITE ACCESS : varmen_db is the default — ask Varmens for credentials
 EVERY TASK : Google Sheet requirement → GPT Project (instructions + requirement + DB-structure file) → discover DB → folder prompt → build → CLAUDE.md/README/handover/evidence/prompts → generate HTML → push to assigned URL → push to GitHub
 FIRST TASKS : GPT-first is MANDATORY. Later: optional, still recommended.
 DAILY LOG : INSERT INTO daily_task.tbl_<project>_<you> (8 fields min)
 TAKING OVER : Clone folder → open Claude Code → ask it to explain (Sec. 11)
 NEVER : share passwords with Claude/GPT, UPDATE old log rows, use test/final/new/temp/old as a folder or file name
 CONTACT : Varmens — varmensk.digitweb@gmail.com